



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

INTELLIGENT VACCINE COLD STORAGE MONITORING SYSTEM
INDUSTRY CONTEXT
CODE REVIEW & REPOSITORY EVALUATION

Submitted in the partial fulfilment for the Course of
CSA1016 – SOFTWARE ENGINEERING

to the award of the degree of
BACHELOR OF TECHNOLOGY IN INFORMATION TECHNOLOGY

Submitted by
SAMEER AHMED G (192421154)

Under the Supervision of
DR ANITA DAVAMANI

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences

Chennai-602105

July 2026

1. PROBLEM OVERVIEW

The Intelligent Vaccine Cold Storage Monitoring System is a software solution designed to monitor vaccine storage conditions and identify unsafe environmental conditions. Vaccine storage requires controlled temperature conditions, and failures in refrigeration, power supply, or environmental monitoring can result in vaccine degradation and operational losses.

The implemented prototype consists of a Flask-based REST API, a browser-based monitoring dashboard, and MongoDB for data persistence. The application provides storage-unit information, temperature and humidity monitoring, temperature-status evaluation, alert information, and database connectivity monitoring.

The project was maintained using Git and GitHub and deployed using Docker and Docker Compose. The purpose of this assessment is to evaluate the quality, organization, maintainability, documentation, version-control practices, and testing of the implemented project.

1.1 Project Objectives

The main objectives of the implemented system are:

- Provide centralized vaccine storage monitoring.
- Process temperature and humidity readings.
- Identify unsafe temperature conditions.
- Store sensor information.
- Provide REST API access.
- Display monitoring information through a dashboard.
- Maintain source-code history using Git.
- Provide containerized deployment using Docker.

1.2 Key Functionalities

The implemented application provides:

1. Storage-unit monitoring.
2. Temperature monitoring.
3. Humidity monitoring.
4. Temperature-status calculation.
5. Critical-condition detection.
6. REST API endpoints.
7. MongoDB integration.
8. Dashboard visualization.
9. Docker-based deployment.

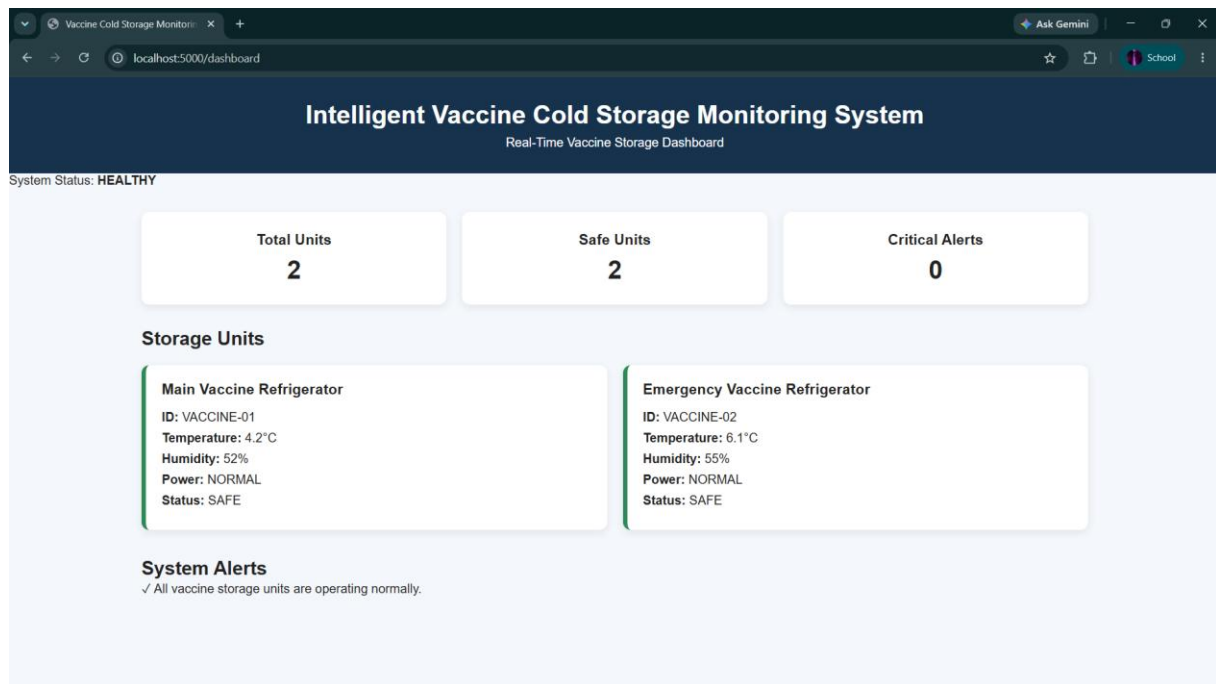


Figure 1: Working Intelligent Vaccine Cold Storage Monitoring Dashboard

2. REPOSITORY ORGANIZATION

The repository follows a modular structure separating backend, frontend, testing, and deployment configuration.

vaccine-cold-storage/

```
|
|
|— backend/
|   |— app.py
|   |— config.py
|   └─ requirements.txt
|
|— frontend/
|   |— index.html
|   |— style.css
|   └─ script.js
|
|— tests/
|   └─ test_api.py
|
|— docker/
|
|— Dockerfile
|— docker-compose.yml
|— .dockerignore
|— .gitignore
└─ README.md
```

The repository structure is logically organized because application logic is separated from presentation files and deployment configuration.

The backend directory contains the Flask API and configuration. The frontend directory contains the user interface. The tests directory provides a dedicated location for testing code. Docker-related configuration is maintained at the repository root.

This organization improves maintainability because developers can identify the location of specific functionality without searching through a single large directory.

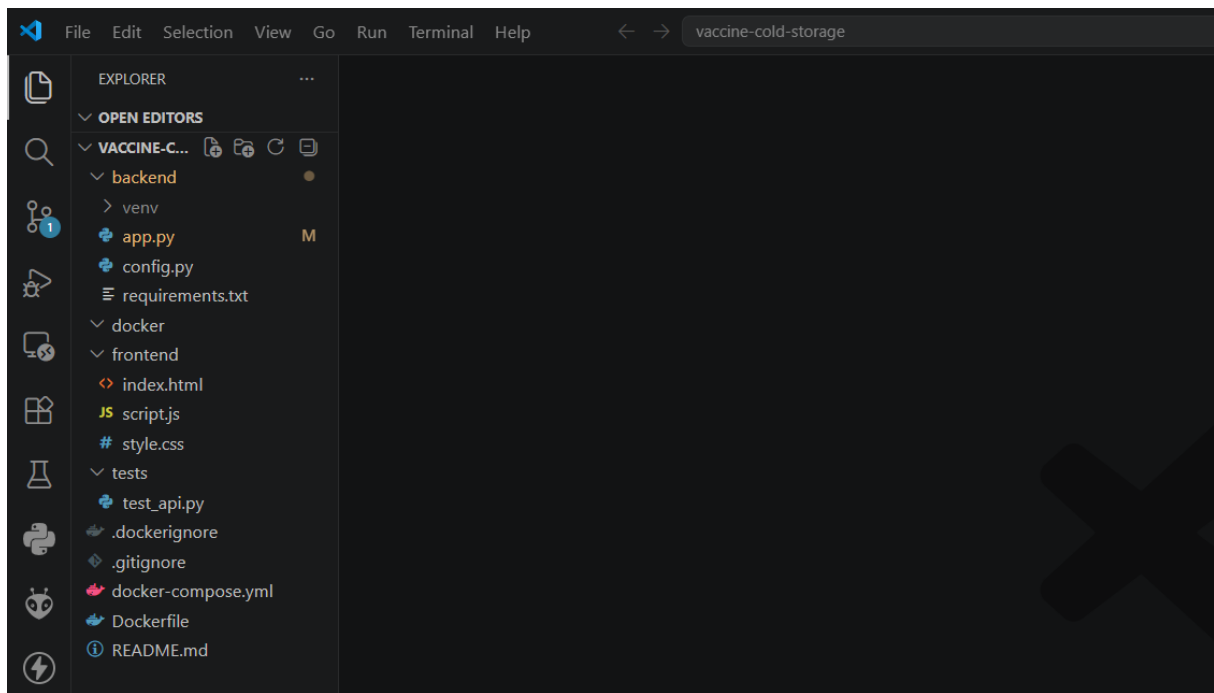


Figure 2: Repository Folder Organization

2.1 Naming Convention Evaluation

The repository generally follows clear and descriptive naming conventions. Files such as `app.py`, `config.py`, `requirements.txt`, `Dockerfile`, and `docker-compose.yml` clearly communicate their purpose.

The use of lowercase names for Python and frontend files also provides consistency across the repository.

Evaluation

Aspect	Observation	Rating
Folder organization	Clearly separated	Good
File naming	Descriptive	Good
Separation of concerns	Backend/frontend separated	Good
Configuration management	Present	Good
Test organization	Dedicated folder	Good
Deployment organization	Docker files clearly identified	Good

3. CODE QUALITY REVIEW

The source code was reviewed using the following criteria:

- Readability
- Modularity
- Naming conventions
- Error handling
- Documentation
- Maintainability
- Reusability
- Separation of concerns

3.1 Backend Code Review

The main backend implementation is located in:

backend/app.py

The application uses Flask to provide REST API endpoints. Functions are separated according to their responsibilities, such as health checking, retrieving storage units, processing sensor readings, and retrieving alerts.

Example:

```
def calculate_status(temperature):  
    if SAFE_MIN <= temperature <= SAFE_MAX:  
        return "SAFE"  
  
    elif temperature > SAFE_MAX:  
        return "CRITICAL"  
  
    return "CRITICAL"
```

The function is small and focused on a single responsibility. It improves readability because the temperature classification logic is not duplicated across multiple API routes.

3.2 API Design

The application provides separate endpoints for different operations.

Endpoint	Method	Purpose
/	GET	Application status
/api/health	GET	Health verification
/api/storage-units	GET	Retrieve storage units
/api/storage-units/<id>	GET	Retrieve specific unit
/api/sensor-reading	POST	Process sensor data
/api/alerts	GET	Retrieve alerts
/api/db-status	GET	Verify database connection

This organization follows a simple REST-oriented approach and makes the backend easier to test and integrate with other applications.

3.3 Error Handling

The backend performs validation when sensor data is submitted.

For example, missing required fields result in an error response:

if unit_id is None or temperature is None or humidity is None:

```
return jsonify({
    "error": "unit_id, temperature and humidity are required"
}), 400
```

This is a positive practice because invalid input is rejected instead of being processed as valid data.

3.4 Code Quality Strengths

The review identified the following strengths:

- Simple and readable Flask implementation.
- Meaningful function names.
- Separate API endpoints.
- Basic input validation.
- Environment-variable support.
- Clear frontend/backend separation.
- Limited duplication.

3.5 Areas for Improvement

Although the code is suitable for a prototype, improvements are required for production deployment.

Issue	Current Condition	Recommendation
Authentication	Not implemented	Add JWT/RBAC
Validation	Basic	Use schema validation
Error handling	Route-level	Centralized error handler
Logging	Limited	Add structured logging
Configuration	Basic	Centralize configuration
Testing	Limited	Expand automated tests
Database layer	Direct access	Introduce repository/service layer
Security	Prototype level	Add HTTPS and security headers

4. VERSION CONTROL EVALUATION

Git was evaluated based on:

- Repository initialization.
- Branching strategy.
- Commit quality.
- Commit history.
- Merging.
- Remote synchronization.

The project uses:

main



This structure demonstrates an organized development workflow.

4.1 Commit History

The repository contains meaningful commits such as:

chore: initialize vaccine monitoring project

feat: add system information API

feat: add dashboard system status

feat: add Docker deployment with MongoDB

These messages are preferable to vague messages such as:

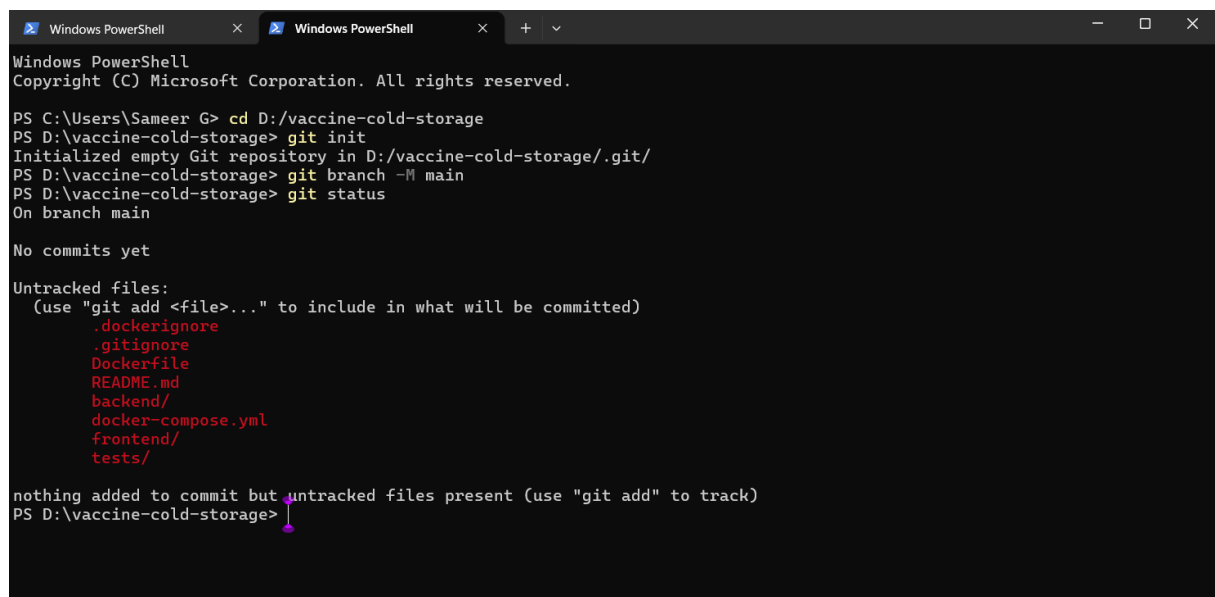
update

changes

final

new code

because they communicate the purpose of each modification.



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Sameer G> cd D:/vaccine-cold-storage
PS D:\vaccine-cold-storage> git init
Initialized empty Git repository in D:/vaccine-cold-storage/.git/
PS D:\vaccine-cold-storage> git branch -M main
PS D:\vaccine-cold-storage> git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .dockerignore
        .gitignore
        Dockerfile
        README.md
        backend/
        docker-compose.yml
        frontend/
        tests/

nothing added to commit but untracked files present (use "git add" to track)
PS D:\vaccine-cold-storage>
```

Figure 3: Git Commit History

4.2 Branching Evaluation

The feature-based branching approach supports collaborative development because individual features can be developed independently and merged after completion.

Evaluation

Git Practice	Evaluation
Main branch	Stable version
Development branch	Integration
Feature branches	Isolated development
Meaningful commits	Good
Merge workflow	Good
Remote repository	GitHub
Repository history	Traceable

```
PS D:\vaccine-cold-storage> git checkout -b develop
Switched to a new branch 'develop'
PS D:\vaccine-cold-storage> git branch
* develop
  main
PS D:\vaccine-cold-storage>
```

5. CODE TESTING AND VALIDATION

Testing was performed using API requests, browser-based verification, database checks, and Docker execution.

5.1 Test Cases

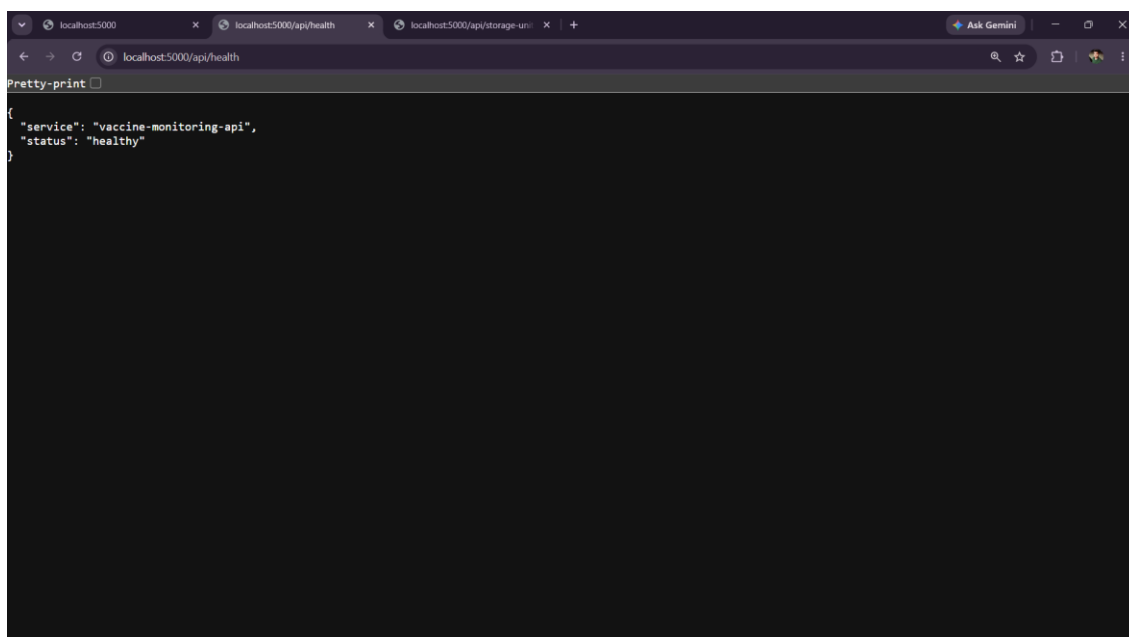
Test ID	Test Case	Expected Result	Result
TC01	Open application root	Application status returned	PASS
TC02	Health API	Healthy status returned	PASS
TC03	Retrieve storage units	Storage-unit data returned	PASS
TC04	Submit safe temperature	Status = SAFE	PASS
TC05	Submit unsafe temperature	Status = CRITICAL	PASS
TC06	Database connection	MongoDB connected	PASS
TC07	Dashboard access	Dashboard loads	PASS
TC08	MongoDB record check	Sensor data available	PASS
TC09	Docker container check	Containers running	PASS
TC10	Container restart	Application recovers	PASS

5.2 API Validation

The health endpoint was tested using:

<http://localhost:5000/api/health>

A successful response confirmed that the Flask backend was operational.

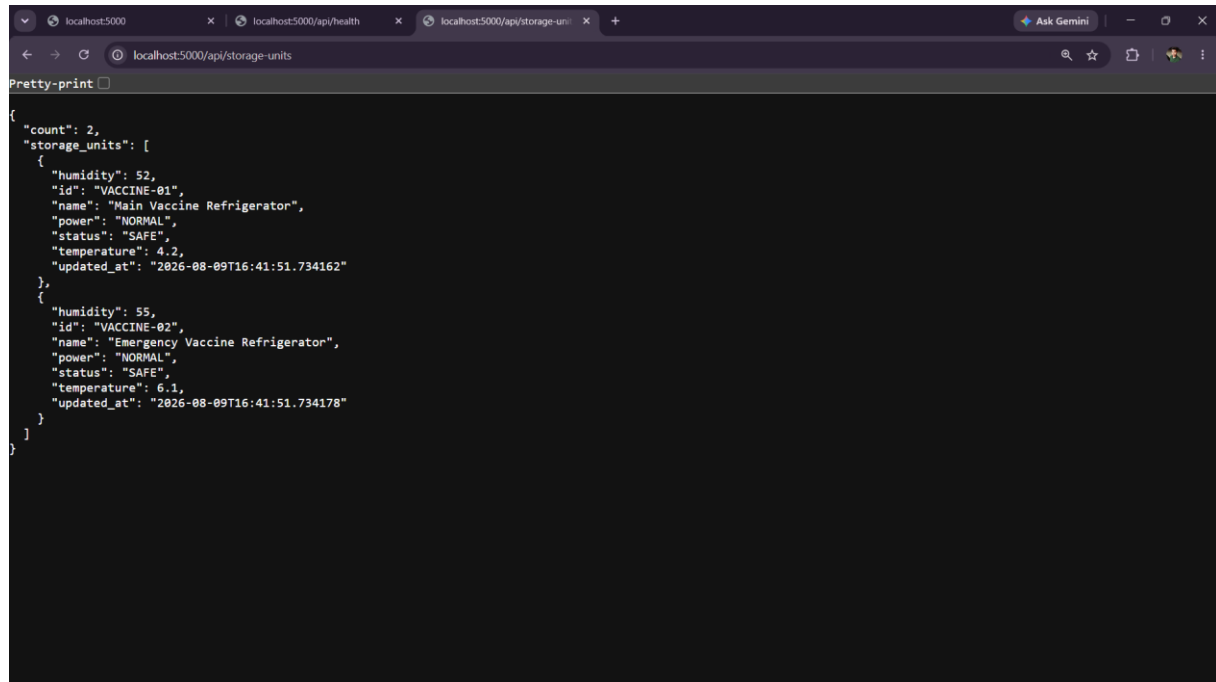


5.3 Temperature Validation

A safe temperature such as **5.2°C** was submitted and classified as **SAFE**.

An unsafe temperature such as **10.5°C** was submitted and classified as **CRITICAL**.

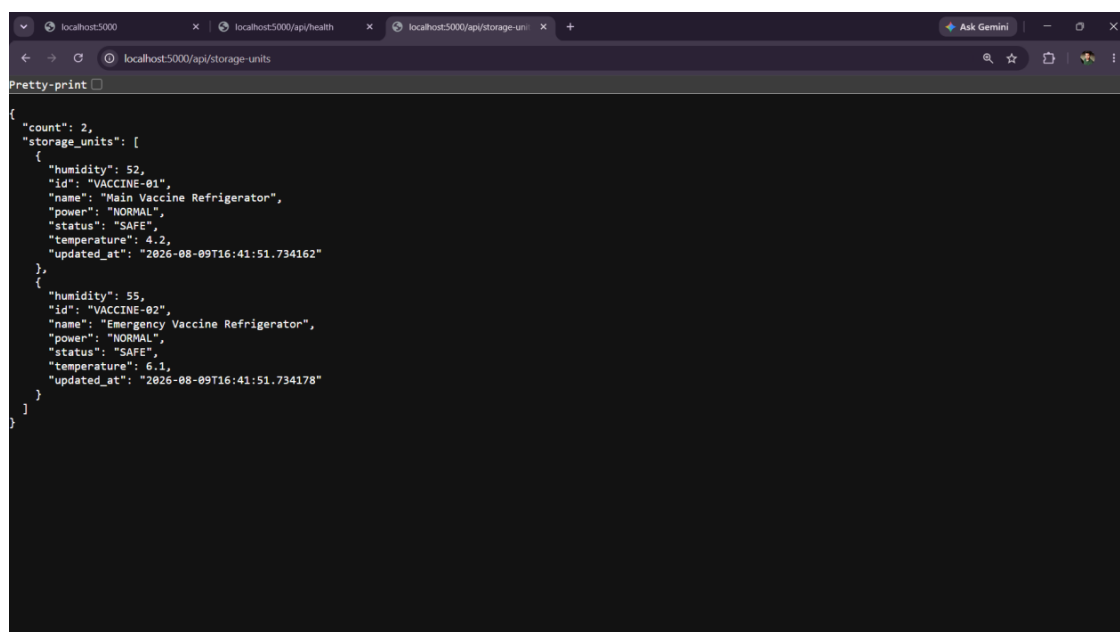
This demonstrates that the application's core temperature-validation logic works correctly.



```
localhost:5000/api/storage-units
Pretty-print
{
  "count": 2,
  "storage_units": [
    {
      "humidity": 52,
      "id": "VACCINE-01",
      "name": "Main Vaccine Refrigerator",
      "power": "NORMAL",
      "status": "SAFE",
      "temperature": 4.2,
      "updated_at": "2026-08-09T16:41:51.734162"
    },
    {
      "humidity": 55,
      "id": "VACCINE-02",
      "name": "Emergency Vaccine Refrigerator",
      "power": "NORMAL",
      "status": "SAFE",
      "temperature": 6.1,
      "updated_at": "2026-08-09T16:41:51.734178"
    }
  ]
}
```

5.4 Database Validation

Sensor information was stored in MongoDB and subsequently verified using the MongoDB shell.

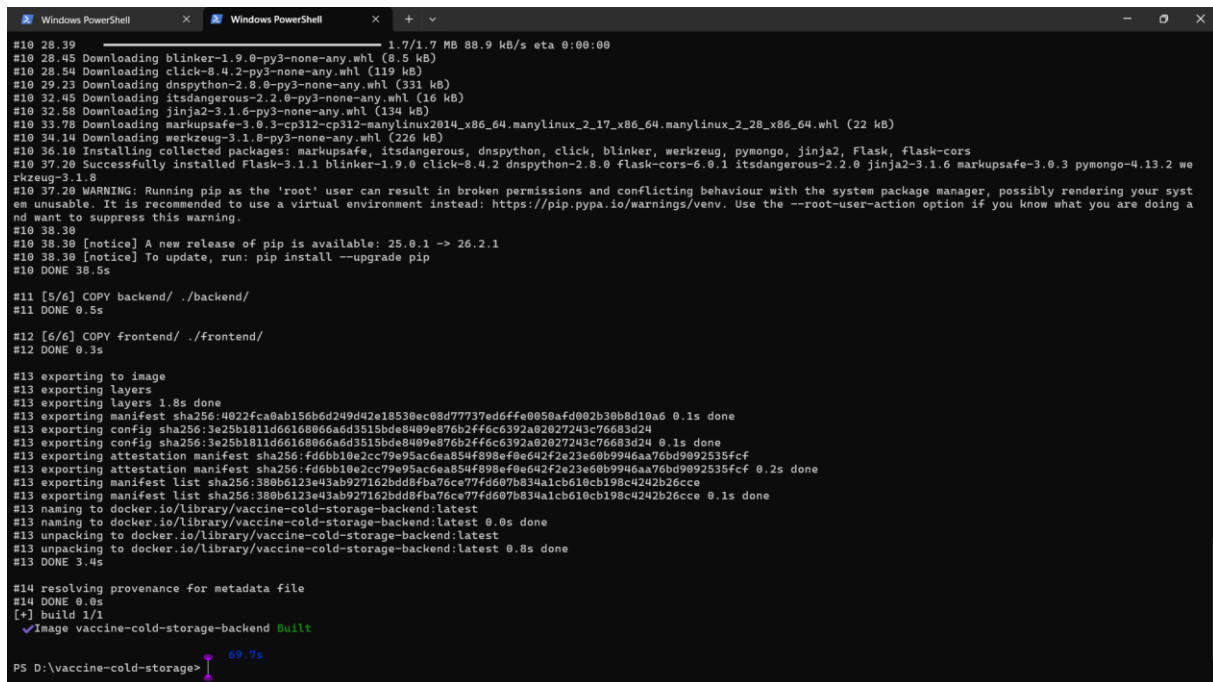


```
localhost:5000/api/storage-units
Pretty-print
{
  "count": 2,
  "storage_units": [
    {
      "humidity": 52,
      "id": "VACCINE-01",
      "name": "Main Vaccine Refrigerator",
      "power": "NORMAL",
      "status": "SAFE",
      "temperature": 4.2,
      "updated_at": "2026-08-09T16:41:51.734162"
    },
    {
      "humidity": 55,
      "id": "VACCINE-02",
      "name": "Emergency Vaccine Refrigerator",
      "power": "NORMAL",
      "status": "SAFE",
      "temperature": 6.1,
      "updated_at": "2026-08-09T16:41:51.734178"
    }
  ]
}
```

5.5 Docker Validation

The application was executed using Docker Compose. Running containers were verified using:

```
docker compose ps
```



```
PS D:\vaccine-cold-storage> docker build -t vaccine-cold-storage-backend .
#10 28.39 --> 1.7/1.7 MB 88.9 kB/s eta 0:00:00
#10 28.45 Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
#10 28.54 Downloading click-8.4.2-py3-none-any.whl (119 kB)
#10 29.23 Downloading dnspython-2.8.0-py3-none-any.whl (331 kB)
#10 32.45 Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
#10 32.58 Downloading Jinja2-3.1.6-py3-none-any.whl (134 kB)
#10 33.78 Downloading markupsafe-3.0.3-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (22 kB)
#10 34.14 Downloading Werkzeug-3.1.8-py3-none-any.whl (226 kB)
#10 36.10 Installing collected packages: markupsafe, itsdangerous, dnspython, click, blinker, Werkzeug, PyMongo, Jinja2, Flask, flask-cors
#10 37.20 Successfully installed Flask-3.1.1 blinker-1.9.0 click-8.4.2 dnspython-2.8.0 flask-cors-6.0.1 itsdangerous-2.2.0 Jinja2-3.1.6 markupsafe-3.0.3 PyMongo-4.13.2 Werkzeug-3.1.8
#10 37.20 WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager, possibly rendering your system unusable. It is recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv. Use the --root-user-action option if you know what you are doing and want to suppress this warning.
#10 38.30
#10 38.30 [notice] A new release of pip is available: 25.0.1 -> 26.2.1
#10 38.30 [notice] To update, run: pip install --upgrade pip
#10 DONE 38.5s

#11 [5/6] COPY backend/ ./backend/
#11 DONE 0.5s

#12 [6/6] COPY frontend/ ./frontend/
#12 DONE 0.3s

#13 exporting to image
#13 exporting layers
#13 exporting layers 1.8s done
#13 exporting manifest sha256:4022fca0ab156b6d249d42e18530ec08d77737ed6ffe0050afd002b30b8d10a6 0.1s done
#13 exporting config sha256:3e25b181d66168066a6d3515bde8409e870b2ff6c6392a02072743c76683d24
#13 exporting config sha256:3e25b181d66168066a6d3515bde8409e870b2ff6c6392a02072743c76683d24 0.1s done
#13 exporting attestation manifest sha256:fd6bb10e2cc79e95ac6ea854f898ef0e642f2e23e60b9946aa76bd9092535fcf
#13 exporting attestation manifest sha256:fd6bb10e2cc79e95ac6ea854f898ef0e642f2e23e60b9946aa76bd9092535fcf 0.2s done
#13 exporting manifest list sha256:380b6123e43ab927162bdd8fba76ce77fd607b834a1cb610cb198c4242b26cce
#13 exporting manifest list sha256:380b6123e43ab927162bdd8fba76ce77fd607b834a1cb610cb198c4242b26cce 0.1s done
#13 naming to docker.io/library/vaccine-cold-storage-backend:latest
#13 naming to docker.io/library/vaccine-cold-storage-backend:latest 0.0s done
#13 unpacking to docker.io/library/vaccine-cold-storage-backend:latest
#13 unpacking to docker.io/library/vaccine-cold-storage-backend:latest 0.8s done
#13 DONE 3.4s

#14 resolving provenance for metadata file
#14 DONE 0.0s
[+] build 1/1
Image vaccine-cold-storage-backend Built
69.7s
PS D:\vaccine-cold-storage>
```

6. REPOSITORY DOCUMENTATION

The repository documentation was reviewed to determine whether a new developer could understand and execute the project without additional assistance.

The README should provide:

- Project description.
- Features.
- Technology stack.
- Requirements.
- Installation procedure.
- Local execution instructions.
- Docker execution instructions.
- API endpoints.
- Repository structure.
- Testing instructions.

6.1 Updated README

Update your existing README.md with the following content:

Intelligent Vaccine Cold Storage Monitoring System

1. Project Overview

The Intelligent Vaccine Cold Storage Monitoring System is a full-stack application designed to monitor vaccine storage conditions. It provides temperature and humidity monitoring, storage-unit status, alert detection, REST APIs, and database storage.

The project demonstrates the application of Git, GitHub, Docker, Docker Compose, Flask, JavaScript, and MongoDB in a software engineering environment.

2. Features

- Vaccine storage-unit monitoring
- Temperature monitoring
- Humidity monitoring
- Safe and critical temperature detection
- Alert monitoring
- REST API
- MongoDB database
- Web monitoring dashboard
- Docker containerization
- Docker Compose deployment

3. Technology Stack

Backend

- Python
- Flask
- Flask-CORS
- PyMongo

Frontend

- HTML
- CSS
- JavaScript

Database

- MongoDB

DevOps

- Git
- GitHub
- Docker
- Docker Compose

4. Project Structure

vaccine-cold-storage/

```
|— backend/  
|— frontend/  
|— tests/  
|— docker/  
|— Dockerfile  
|— docker-compose.yml  
|— .gitignore  
|— .dockerignore  
└— README.md
```

5. Running Locally

Install Python 3.12 or later.

Create a virtual environment:

```
python -m venv venv
```

Activate the environment and install dependencies:

```
pip install -r backend/requirements.txt
```

Run the application:

```
python backend/app.py
```

Open:

```
http://localhost:5000
```

6. Running with Docker

Build the application:

```
docker compose build
```

Start the services:

```
docker compose up -d
```

Check running containers:

```
docker compose ps
```

Stop the services:

```
docker compose down
```

7. API Endpoints

```
GET /
```

```
GET /api/health
```

```
GET /api/storage-units
```

```
GET /api/storage-units/<unit_id>
```

```
POST /api/sensor-reading
```

```
GET /api/alerts
```

```
GET /api/db-status
```


8. Temperature Monitoring

The prototype uses a safe vaccine temperature range of 2°C to 8°C.

Values outside the configured range are classified as critical.

9. Git Workflow

The project uses:

main

develop

feature/backend-api

feature/frontend-dashboard

feature/docker

Feature branches are merged into develop and the tested version is merged into main.

10. Testing

Testing includes:

- API health testing
- Storage-unit API testing
- Sensor-reading testing
- Temperature validation
- MongoDB connectivity
- Dashboard validation
- Docker container testing

11. Future Enhancements

Future versions can integrate real IoT sensors, machine learning-based equipment failure prediction, authentication, mobile notifications, cloud deployment, CI/CD, and advanced analytics.

After updating the README, take a screenshot.

GitHub - Samoor007/Vaccine-Cold-Storage

README

Intelligent Vaccine Cold Storage Monitoring System

An IoT-ready full-stack prototype for monitoring vaccine storage conditions, detecting temperature excursions, and maintaining sensor data securely.

Features

- Real-time temperature monitoring
- Humidity monitoring
- Safe/Critical temperature detection
- Vaccine storage-unit monitoring
- Web-based monitoring dashboard
- REST API
- MongoDB data storage
- Docker & Docker Compose deployment
- Git & GitHub version control

Tech Stack

Category	Technologies
Backend	Python, Flask
Frontend	HTML, CSS, JavaScript
Database	MongoDB
Version Control	Git, GitHub
Containerization	Docker, Docker Compose

Publish your first package

Contributors 1

Samoor007 Sameer Ahmed G

Languages

Python 45.1%

JavaScript 28.3%

CSS 12.6%

HTML 11.8%

Dockerfile 2.2%

Suggested workflows

Based on your tech stack

Webpack

Build a NodeJS project with npm and webpack. By GitHub Actions

Configure

Pylint

Lint a Python application with pylint. By GitHub Actions

Configure

SLSA Generic generator

Generate SLSA3 provenance for your existing release workflows. By Open Source Security Foundation (OpenSSF)

Configure

[More workflows](#)

Page | 18

7. CODE REVIEW FINDINGS AND RECOMMENDATIONS

The overall review indicates that the project has a clear modular structure, readable source code, an organized Git workflow, containerized deployment, and basic testing capabilities.

However, the current implementation should be considered a **prototype rather than a production-ready healthcare application**.

7.1 Major Findings

Area	Finding	Priority
Code organization	Good separation of frontend/backend	Low
Readability	Code is easy to understand	Low
API design	Basic REST structure	Medium
Validation	Basic input validation	High
Authentication	Not implemented	Critical
Testing	Basic testing	High
Documentation	Improved README required	Medium
Git workflow	Well organized	Low
Docker	Proper containerization	Low
Database	Functional MongoDB integration	Medium
Security	Requires strengthening	Critical

7.2 Recommendations

1. Improve Authentication

Authentication and role-based access control should be implemented before production use. Healthcare administrators, nurses, pharmacists, and other users should have appropriate permissions.

2. Improve Input Validation

A structured validation library should be used to validate temperature, humidity, storage-unit IDs, and other incoming data.

3. Improve Testing

The project should eventually include automated unit, integration, and end-to-end tests.

4. Improve Security

HTTPS, secure credentials, authentication, authorization, secret management, and security headers should be implemented.

5. Improve Logging

Structured application logs should be introduced to help diagnose API failures and equipment-monitoring issues.

6. Improve Architecture

As the system grows, monitoring, alerting, inventory, AI prediction, and reporting can be separated into independent services.

7. Add CI/CD

GitHub Actions can automatically execute tests and build Docker images whenever changes are pushed.

8. OVERALL EVALUATION

Evaluation Area	Rating
Repository Organization	Good
Code Readability	Good
Modularity	Good
Version Control	Very Good
Commit Practices	Very Good
Testing	Good
Documentation	Good after README update
Docker Implementation	Very Good
Maintainability	Good
Production Security	Needs Improvement
Overall Prototype Quality	Good

The project demonstrates the fundamental practices expected from a collaborative software engineering project. Its repository organization, Git workflow, Docker deployment, and basic testing provide a strong foundation. The major remaining improvements concern production-level security, automated testing, authentication, validation, logging, and scalability.

9. CONCLUSION

The Code Review and Repository Evaluation of the Intelligent Vaccine Cold Storage Monitoring System demonstrates that the project follows a structured software engineering approach. The repository is logically organized, the source code is reasonably readable, and the Git workflow provides traceable development history.

The use of meaningful commits and feature branches supports collaborative development, while Docker and Docker Compose provide a consistent deployment environment. Testing through API requests, dashboard verification, database checks, and container execution provides evidence that the implemented prototype operates correctly.

The review also identified areas requiring improvement, particularly authentication, security, automated testing, centralized error handling, logging, and production-level validation. Addressing these issues would improve the maintainability, reliability, and readiness of the system for real healthcare deployment.

10. REFERENCES

- [1] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2016.
- [2] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed. McGraw-Hill, 2020.
- [3] Git, “Git Documentation.” [Git Documentation](#)
- [4] GitHub, “GitHub Documentation.” [GitHub Documentation](#)
- [5] Docker, “Docker Documentation.” [Docker Documentation](#)
- [6] Docker, “Docker Compose Documentation.” [Docker Compose Documentation](#)
- [7] M. Fowler, “Continuous Integration.” [Continuous Integration](#)
- [8] M. Kleppmann, *Designing Data-Intensive Applications*. O'Reilly Media, 2017.